AUTOMAÇÃO DE TESTES WEB

Playwright: do locator à evidência

Monte um teste, acompanhe o estado da interface e investigue uma falha antes de decidir se o problema está no produto ou no teste.

O caso Fikily

Selecionar Acessórios, buscar Aurora e aplicar deve produzir uma lista pronta com Mochila Aurora e Estojo Aurora. Teclado Aurora é um eletrônico e não deve aparecer. Todos os dados são fictícios.

Prática	O que muda neste módulo
Construir	Estrutura async/await, locators e assertions web-first.
Observar	Prontidão do catálogo e snapshots em uma linha do tempo didática.
Investigar	Referência aderente, defeito na candidata e teste frágil.
Revisar	Estado conhecido, isolamento, retries e parecer limitado ao escopo.

Simulação não é execução real

O QA Quest não instala nem executa Playwright, código do aluno ou serviços externos. Os resultados são preparados localmente. O exemplo requer um projeto Playwright Test e uma aplicação de teste autorizada configurados fora do site.

A leitura é opcional e não altera XP. Fundamentos e Cypress são recomendações de estudo, não pré-requisitos de acesso além da Fase 4.

01 / ESTRUTURA E LOCATORS

Descreva o elemento que você procura.

test e expect vêm de @playwright/test. test.describe agrupa casos; beforeEach prepara cada caso. async/await organiza operações assíncronas; Arrange, Act, Assert ajuda a separar preparação, ações e verificações. [1]

```
import { test, expect } from "@playwright/test";
test.describe("Catálogo Fikily", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/catalog");
  });
  test("filtra acessórios", async ({ page }) => {
    const category = page.getByLabel("Categoria");
    await category.selectOption("accessories");
  });
});
```

/catalog é uma rota de exemplo: exige baseURL e uma aplicação que implemente esse caminho. Não é uma URL pública oferecida pelo QA Quest.

Locator	Contrato e cuidado
getByRole	Papel e nome acessível: button com nome Aplicar; searchbox com nome Buscar produto.
getByLabel	Rótulo associado ao campo: Categoria.
getByTestId	Atributo explícito mantido pelo time: catalog-results.
getByPlaceholder	Placeholder Ex.: Aurora, quando esse texto é a intenção da consulta.

Locator não é um elemento congelado

Ao ser utilizado, o locator procura um elemento atual. CSS e XPath também podem ser válidos; o risco é acoplar o teste a cor ou posição sem intenção. Nenhuma dessas consultas substitui uma avaliação completa de acessibilidade. [2]

02 / AÇÕES E ASSERTIONS

O resultado precisa corresponder à regra.

Ações aguardam condições de actionability aplicáveis, não o sucesso do negócio. As verificações variam entre selectOption, fill e click. O exemplo abaixo usa a massa conhecida de dois acessórios. [3, 4]

```
import { test, expect } from "@playwright/test";
test.describe("Catálogo Fikily", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/catalog");
  });
  test("filtra acessórios Aurora", async ({ page }) => {
    const category = page.getByLabel("Categoria", { exact: true });
    const search = page.getByRole("searchbox", { name: "Buscar produto", exact: true });
    const apply = page.getByRole("button", { name: "Aplicar", exact: true });
    const results = page.getByTestId("catalog-results");
    await category.selectOption("accessories");
    await search.fill("Aurora");
    await apply.click();
    await expect(results).toHaveAttribute("data-state", "ready");
    const cards = results.getByTestId("product-card");
    await expect(cards).toHaveCount(2);
    await expect(cards).toContainText(["Mochila Aurora", "Estojo Aurora"]);
    for (let index = 0; index < 2; index++) {
      await expect(cards.nth(index))
        .toHaveAttribute("data-category", "accessories");
    }
  });
});
```

toHaveCount(2) impede lista vazia nesta massa. toContainText confere nomes; toHaveAttribute confere a categoria dos dois cards esperados. Dois itens com um eletrônico ainda violam o contrato, mesmo se a contagem passar. [4, 5]

03 / SINCRONIZAÇÃO E EVIDÊNCIAS

Leia o estado antes de atribuir a causa.

Prefira uma assertion sobre uma condição observável, como data-state=ready. Uma pausa fixa não prova que a atualização terminou. networkidle não é a recomendação geral para comprovar prontidão de testes. [4, 6]

Perfil	Esperado e observado	Interpretação e impacto
Referência confiável	Esperado: lista pronta só com acessórios. 800 ms: dois acessórios; verificações passam.	Caso aderente, somente no escopo preparado.
Candidata confiável	Esperado: nenhum eletrônico. 800 ms: lista ready com três produtos, um eletrônico.	Defeito do produto. A pessoa recebe resultado fora do filtro.
Referência frágil	Esperado: conferir após atualização. 300 ms: snapshot antigo de três itens. 800 ms: lista correta com dois acessórios.	Falha do teste: falso alarme causado por leitura prematura.

```
await page.waitForTimeout(300);
const snapshot = await page.getByTestId("product-card").count();
expect(snapshot).toBe(2);
```

O número capturado não muda sozinho. Essa assertion imediata difere de await expect(cards).toHaveCount(2), que pode reconsultar. expect.poll reavalia uma função até um critério passar; não é, por si, a causa de instabilidade. [4]

Limites desta evidência

Tempos e eventos são fixtures determinísticas, não métricas do computador. A linha do tempo não é o Trace Viewer real. waitForResponse só faz sentido quando há uma resposta relevante; este laboratório não faz requisições. [6]

04 / REVISÃO E PARECER

Independência também é parte da qualidade.

Playwright Test fornece contextos de navegador isolados por teste. Fixtures oferecem recursos; beforeEach pode preparar o cenário. Projects são configurações e storageState pode reutilizar autenticação: nenhum garante independência sozinho. [7, 8]

Retry da assertion tenta satisfazer uma condição; retry do teste executa uma nova tentativa do caso. No projeto real, trace na primeira nova tentativa exige retries habilitados. O QA Quest não gera ou recebe trace.zip. [9]

```
import { defineConfig } from "@playwright/test";
export default defineConfig({
  retries: 1,
  use: { trace: "on-first-retry" },
});
```

Parecer sustentado

A referência atende ao caso preparado. A candidata precisa de ajuste no filtro e nova avaliação. O teste frágil precisa de sincronização por condição. Concluir corretamente a missão não equivale a aprovar uma suíte real ou a aplicação para produção.

Exercício de revisão

1. Por que o clique concluído não comprova a atualização?
2. Dois cards garantem que o filtro está correto?
3. Qual a diferença entre snapshot e assertion web-first?
4. Por que beforeEach e storageState não substituem independência?

Critérios de correção

1. A ação pode terminar antes do resultado de negócio.
2. Não; nomes e categoria também precisam corresponder ao contrato.
3. O snapshot captura um valor; a assertion sobre locator pode reconsultar.
4. O caso precisa de estado controlado e não deve depender de outro teste.

05 / REFERÊNCIAS E PRÓXIMA PRÁTICA

Continue com fontes gratuitas.

Conteúdo autoral didático, com exemplos próprios e paráfrases. Consulte a documentação da versão usada no seu projeto. As fontes abaixo foram verificadas em 05/11/2026.

[1] **Estrutura e escrita de testes**
https://playwright.dev/docs/writing-tests

[2] **Locators e contratos**
https://playwright.dev/docs/locators

[3] **Auto-waiting e actionability**
https://playwright.dev/docs/actionability

[4] **Assertions, retry e polling**
https://playwright.dev/docs/test-assertions

[5] **Assertions de locators**
https://playwright.dev/docs/api/class-locatorassertions

[6] **Page: carregamento e respostas**
https://playwright.dev/docs/api/class-page

[7] **Isolamento por contextos**
https://playwright.dev/docs/browser-contexts

[8] **Fixtures**
https://playwright.dev/docs/test-fixtures

[9] **Trace Viewer e retries**
https://playwright.dev/docs/trace-viewer

StepScript: opcional e sem XP extra

Projeto complementar de Gustavo Favero, criador do QA Quest. Permite gravar interações e gerar código Cypress/Playwright, mas não executa os testes pelo painel. Grave apenas ambientes autorizados com dados fictícios. Revise valores, seletores, esperas e assertions antes de compartilhar. O QA Quest não recebe suas gravações nem exige a extensão.

StepScript na Chrome Web Store

Versão 1.0 · 05/11/2026. A execução real é um assunto externo ao laboratório local. Nenhuma instalação é necessária para concluir esta missão.